Conditional Data Flow Analysis

Elena Sherman
Department of Computer Science and Engineering
University of Nebraska - Lincoln
Lincoln, Nebraska 68588-015
Email: esherman@cse.unl.edu

Matthew B. Dwyer
Department of Computer Science and Engineering
University of Nebraska - Lincoln
Lincoln, Nebraska 68588-015
Email: dwyer@cse.unl.edu

***Abstract*—The abstract goes here.**

I. Introduction

The degree to which a software must satisfy its requirements depends on the criticality of the software. An undetected fault in critical software may result in significant financial losses or human fatalities, e.g., a fault in a flight control system. Thus, the degree of the conformance, i.e., an adequacy criterion, for critical software is set at the highest level – software must comply with its requirements on all possible inputs. Yet, for the majority of applications adequacy criteria are more relaxed, e.g., to be 85% statement adequate, a property must hold on a set of inputs that executes 85% of the application's statements.

To accommodate these different levels of compliance, software engineers have developed various techniques best suitable for a specific criterion. Mainly these techniques are classified into dynamic and static analyses. Dynamic analysis, such as testing, verifies that a program is adequate by executing it. This approach is widely used for non-critical software that uses weaker adequacy criteria. Employing testing to check a program's conformance on all inputs is usually intractable. Static analysis checks the conformance of a program to its property by analyzing the program code, i.e., without executing it. This compile-time determination allows static analysis to consider all possible program behaviors, which is suitable for checking program conformance to its properties at the highest level. Hence, static analysis is a key technique for verifying critical systems.

The power of static analysis to consider all program behaviors follows from its ability to safely approximate program behaviors by abstracting the concrete domain of program variables and the programming language semantics. But at the same time because of its over-approximating nature, static analysis may identify some property violations as uncertain. The reason for this uncertainty is that a static analysis cannot tell if a violation happens on a feasible or an infeasible program behavior. This inconclusiveness is unacceptable for critical system and thus each potential violation must be examined further. An automatic solution to the elimination of false positive violations is to increase the precision of a static analysis, i.e., improve the analysis so it considers fewer infeasible behaviors.

One way to improve the precision of a static analysis is to refine its abstract domain and program semantics interpretation which is always theoretically possible [1], or to combine different static analyses in a suitable manner as discussed in [2]. Besides the laborious effort of redesigning a static analysis or combining two of them together, the resulting analysis usually becomes expensive in terms of time and memory consumption which makes it inapplicable to large applications. Thus precision and scalability of a static analysis work against each other. When a static analysis has a high precision generally it is not scalable and conversely when a static analysis is scalable its precision typically suffers. The literature suggests two complementary ideas for approaching this dilemma.

One approach focuses on making a precise static analysis scalable by partitioning a large program into modules like procedures and classes, and allowing the analysis to examine each partition independently. Next, the analyzed information of each module is composed together to obtain the result of the whole program analysis. In the literature [3], [4] this method is referred to as *partial static analysis*.

Another approach concentrates on improving the precision of a scalable analysis by permitting the analysis to explore only those program states for which it is adequately precise, i.e., able to provide definitive result. In the literature [5], [6], [7] this approach is called *conditional static analysis* since the permitted states are described by a condition θ. In this framework an analysis verifies a program under some assumptions, i.e., there is no null pointer exception in the program. Next, another analysis attempts to prove these assumptions by either showing that the states which do not satisfy θ are not reachable or do not lead to property violations. In the previous work the condition θ either determined from the analysis design [5], [6] when θ is applicable to all program states or determined during program analysis execution [7] when θ is composed from the conditions assumed to be held for a certain set of states.

Our work focuses on the latter approach applied to data-flow analysis, i.e., conditional data-flow analysis. Unlike the cases presented in the previous work, data-flow analysis cannot determine a priori or during its execution the conditions under which it is able to verify a program to its best abilities. Moreover, data-flow analysis has two alternative ways for expressing θ. The main goal of our work is to explore different approaches in θ encoding for data-flow analysis so that a scalable analysis is able to optimize its result. In the next

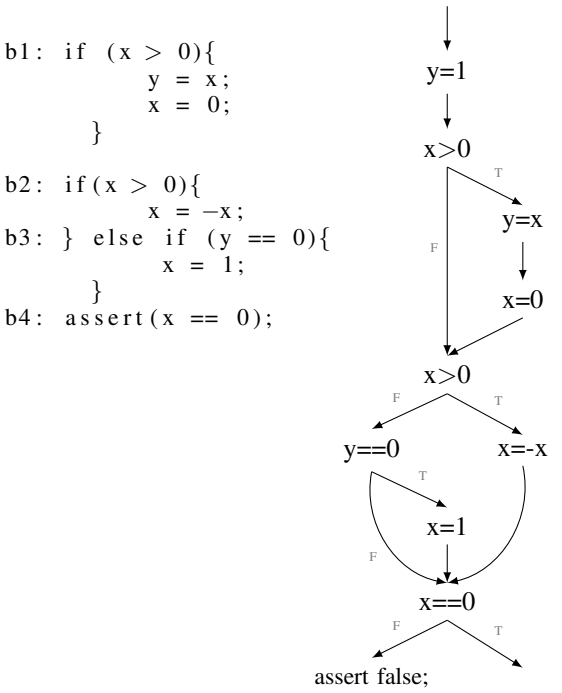

Fig. 1. Program example: source code and the corresponding CFG

section we illustrate the concept of conditional data-flow analysis, different encodings for θ and their effect. In the next section we also present our research questions and the outline for the rest of the paper.

II. Overview

Before introducing conditional data-flow analysis we would like to start with an example of a traditional data-flow analysis. Data-flow analysis calculates some information for each point in a program based on the program structure and the language semantics. The calculated facts then later used to reason about program properties. One type of property that we consider in this work is safety property. A safety property φ is a fact about the program that must hold on all feasible program executions. The type of data-flow analyses that able to determine properties that are satisfied by all paths are called *must* analyses. Commonly used safety properties are program assertions and we going to assume them in our further discussions.

Consider a program and its corresponding control flow graph (CFG) on Figure 1. In this example x and y are variables of the integer type and the edges of CFG are labeled with T for *true* branches and F for *false* branches of conditional statements. In this example we want to use a data-flow analysis to verify that the assertion at the label b_4 holds. In CFG we "desugare" the assertion into a conditional statement.

To verify the assertion we employ *zero* data-flow analysis whose abstract domain is defined by the set $\{\bot, 0, \neg 0, \top\}$. Where $\bot$ element represents the empty set of concrete values; 0 – the singleton set $\{0\}$ of concrete values, $\neg 0$ – any concrete value except 0 and $\top$ – any concrete value. The left CFG on Figure 2 shows the result of zero analysis where each edge is annotated with a tuple containing abstract values for x and y variables respectively. Clearly the analysis fails to verify φ since it cannot prove that the false branch of x==0 condition is infeasible, i.e., the value of x or y is not $\bot$.

One of the main reasons why data-flow analysis fails to verify φ is the over-approximation that takes place when variable values from different paths are merged. For example, before the merge point the value of x on the true branch of b_1 is 0 which is safely over-approximated by $\top$ at the merge point with the false branch of b_1. Thus, to improve the precision of data-flow analysis we need to restrict the set of paths it analyzes. It can be done by either restricting entry values of variables or by explicitly specifying the set of paths to be analyzed. We refer to the former as predicate-based condition and the latter as path-based condition.

To illustrate conditional data-flow analysis with predicate-based we use the previous zero analysis and restrict the entry value of x to the abstract domain element 0. The result of such conditional analysis is shown on middle left CFG of Figure 2. The picture shows that condition was able to increase the precision of zero analysis since the analysis was able to verify the program for the entry value x$= 0$.

Obviously in order to make the verification unconditional the analysis must also show that φ also holds for the condition complement, i.e., when they entry value of x is $\neg 0$. The middle right CFG that on the same figure shows that as in the case of the original analysis the result is inconclusive. Thus conditions expressed as predicates over the input values not necessarily improve data-flow analysis precision.

To illustrate conditional data-flow analysis with path-based condition we restrict zero analysis to propagate program facts along the true branch of b_1 while skipping its false branch. The right CFG on Figure 2 depicts the result of such conditional analysis. Once again conditional zero analysis was able to determine that all program paths following the true outcome of b_1 do not violate φ. Yet when the analysis is restricted to only the false branch of b_1 the result becomes inconclusive again.

These examples indicate that some predicate and path based conditions posed on data-flow analysis can improve its precision. However, unlike previous work where conditions enabling the best precision can either be determined from the analysis design or during analysis run, for data-flow analysis such conditions cannot be obtained using methods described in the related work. The section on related work describes in details the reason but in the essence the cause is that data-flow analysis cannot distinguish the source of precision losses which can be due to the over-approximation or due to the analysis design.

Thus the purpose of our work is to investigate what type of conditions, i.e., predicate-based, path-based or their combination, is better fitted to improve the precision of data-flow analysis. In order to obtain a better evaluation we not only compare condition types based on their ability to enable the best precision (like on the middle left and right CFGs of Figure 2) but also based their partial result, i.e., when

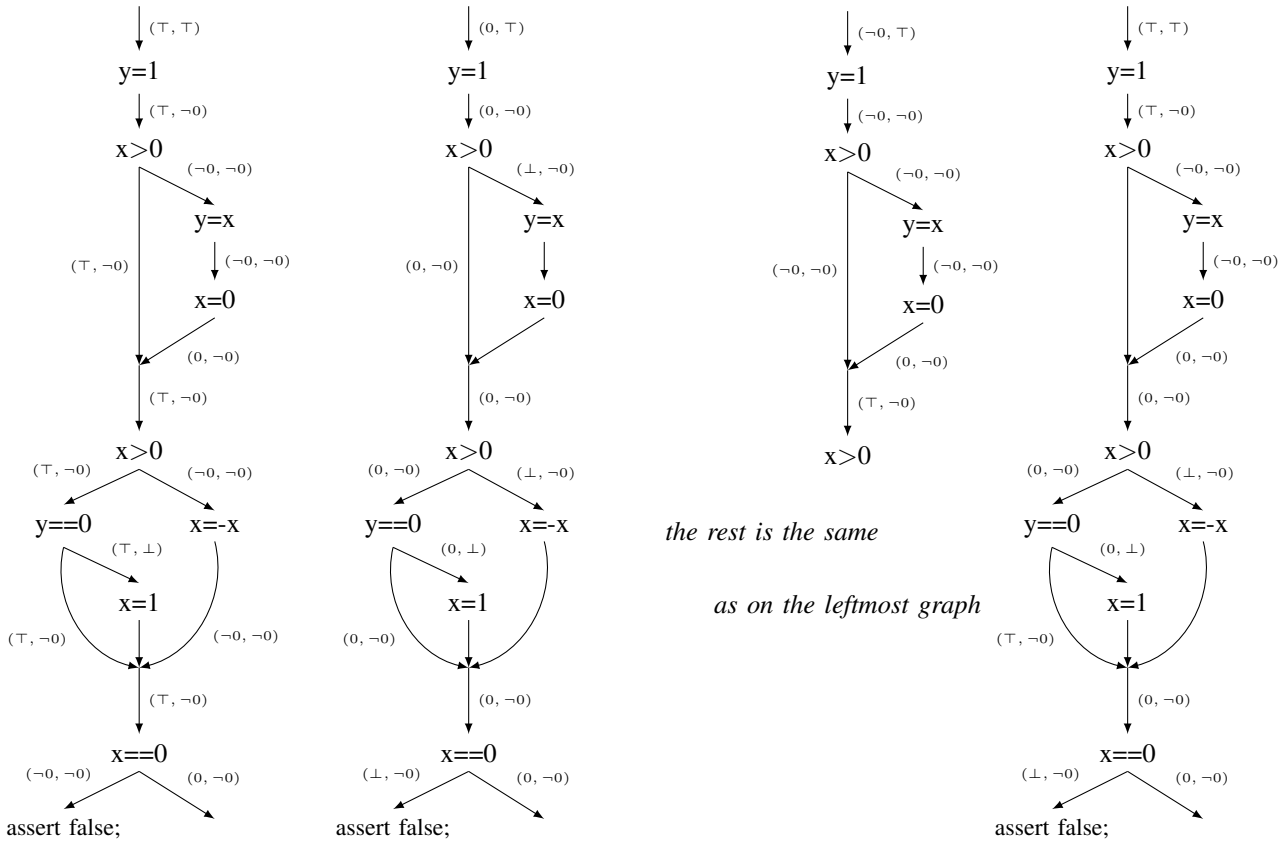

Fig. 2. Analysis examples: *zero* analysis result (left), conditional *zero* analysis result with restricted input domain $x = 0$ (middle left), $x = \neg 0$ (middle right), and the restricted set of paths (right).

an analysis can verify φ on a subset of analyzed paths. As example consider the original zero analysis on left of Figure 2. Even though the analysis cannot verify φ for all paths it can definitely tell that the set of paths containing the true branch of b_3 cannot violate φ because these paths are infeasible. To represent and compare partial results of conditional data-flow analysis we introduce path language. Using such encoding we can record the partial result of unconditional zero analysis as $*; b_3; *$ and the result of condition zero analysis on the rightmost CFG of Figure 2 as $b_1; *$. Section ... explores the paths language in greater details.

Overall our work focuses on answering (empirically) the following research questions:

1) Does the predicate-based precondition increase the partial output of data-flow analysis?
2) Does the path-based precondition increase the partial output of data-flow analysis?
3) How do the two representations compare with each other?
4) Does the combination of predicate-based and path-based preconditions increase the partial output of data-flow analysis comparing to their separate results.

III. Formalizing Conditional Analyses

Similarly to the previous section in this section we first present the traditional monotone framework for data flow analysis followed by the discussion of the necessary changes that extend it to a conditional data flow framework. This section also outlines the approach of composing the unconditional result from conditional ones.

We use the data flow analysis framework as presented in [8] where an analysis $\mathcal{A}$ is defined by the following parameters.

- The complete lattice $D_{\mathcal{A}}$ that describes the abstract domain of $\mathcal{A}$.
- CFG_P for a program P.
- A set of monotone transfer functions $\mathcal{F}_{\mathcal{A}}$ for each statement $l \in CFG_P$ that maps an element of $D_{\mathcal{A}}$ to itself, i.e., $f_l \in \mathcal{F}_{\mathcal{A}} : D_{\mathcal{A}} \mapsto D_{\mathcal{A}}$.
- Entry statements E in CFG_P.
- An initial value $\iota \in D_{\mathcal{A}}$ for statements in E.

Then the set of equations for forward $\mathcal{A}$ is defined as follows on entry and exit of each statement $l \in CFG_P$:

$$\begin{aligned} \mathcal{A}_{in}(l) &= \bigsqcup\{\mathcal{A}_{out}(l') \mid (l', l) \in CFG_P\} \sqcup \iota^l_E \\ &\quad \text{where } \iota^l_E = \begin{cases} \iota & \text{if } l \in E \\ \bot & \text{if } l \notin E \end{cases} \\ \mathcal{A}_{out}(l) &= f_l(\mathcal{A}_{in}(l)) \end{aligned}$$

where $\sqcup$ is the least upper bound operator, $\bot$ is the bottom element of $D_{\mathcal{A}}$ for which $\forall d \in D_{\mathcal{A}} : \bot \sqcup d = d$ and $\forall l \in CFG_P : f_l(\bot) = \bot$. For the safety properties $\bot$ corresponds to the empty set of concrete values and $\top$ to the set containing all concrete values. The value of ι is assigned to $\top$, i.e., the analysis considers all possible input values for a program. The solution of the above set of equations provides is the result of the analysis for P.

Recall that a condition for data flow analysis can be expressed as a predicate ϕ over the input values or as π – condition that identifies the set of paths to be analyzed. Since ϕ and π are incomparable the overall condition θ for a conditional data flow analysis $\mathcal{A}^{\theta}$ is described by a tuple of ϕ and π, i.e., $\mathcal{A}^{(\phi,\pi)}$.

We have chosen π to be represented by the set of branch edges in CFG_P, at most one for each conditional statement l, which the analysis must include while excluding their counterparts. If l has l' and l'' as its true and false targets, respectively, then π can contain the edge (l, l'), or the edge (l, l''), or none of them. To capture the relation between the opposite branches of l we designate $(l, l') = \neg(l, l'')$ and the vice versa $(l, l'') = \neg(l, l')$. If $(l, l') \in \pi$ then the values of all variables x_i incoming to the target of its opposite edge l'', i.e., along edge $\neg(l, l')$, will be set to $\bot$. For brevity we denote such case, i.e., when $\forall i : x_i = \bot$, as $\bot$ state.The same principle applies when $(l, l'') \in \pi$. When none of the edges are present in π then the analysis treats them in its usual manner, i.e., propagates the information through both branches.

For a set of input variables x_i ϕ can be represented as an assignment of x_i to its initial values from $d_i \in D_{\mathcal{A}} \setminus \{\bot\}$. In the traditional data flow analysis all x_i are set to $\top$ element of $D_{\mathcal{A}}$. For shortness we refer to such condition as $\top$.

Thus a traditional data flow analysis $\mathcal{A} = \mathcal{A}^{(\top,\emptyset)}$, for a conditional analysis with predicate-based condition $\mathcal{A}^{(\phi,\emptyset)}$ and so on. In the related work on the conditional model checking a condition θ also consists of two parts. In the case when a model checker runs out of resources it outputs the set of predicates describing analyzed paths, which is equivalent to ϕ predicate-based condition. In the case when a model checker cannot reason about some set of paths it records them in a structural way which is similar to π path-based condition.

With the proposed above predicate and path based conditions we can now write the set of equations for conditional data flow framework for an analysis $\mathcal{A}^{(\phi,\pi)} = \mathcal{A}^{\theta}$:

$$\begin{aligned}
\mathcal{A}^{\theta}_{in}(l) &= \bigsqcup\{\mathcal{A}^{\theta}_{out}(l') \mid (l', l) \in CFG_P, \neg(l', l) \notin \pi\} \sqcup \iota^{l}_{E} \\
&\quad \text{where } \iota^{l}_{E} = \begin{cases} \phi & \text{if } l \in E \\ \bot & \text{if } l \notin E \end{cases} \\
\mathcal{A}^{\theta}_{out}(l) &= f_l(\mathcal{A}^{\theta}_{in}(l))
\end{aligned}$$

where for the first equation we assume $\forall d \in D_{\mathcal{A}} : \emptyset \sqcup d = d$.

Let Φ and Π be the universe of predicate-based and path-based conditions, respectively, for an analysis $\mathcal{A}$. Executing $\mathcal{A}$ with different conditions $\phi_i \in \Phi$ and $\pi_j \in \Pi$ produces a set of conditional analysis $\mathcal{A}^{(\phi_i,\pi_j)}$. Having these conditional analysis we would like to determine whether its combined effort produces the result of the unconditional analysis. To answer this question we first identify the method for combining conditions (ϕ_i, π_j) for the same $\mathcal{A}$ and then establish the requirement when combined conditions imply the unconditional result.

Consider two conditions $(\top, \{(l, l')\})$ and $(\top, \{\neg(l, l')\})$. The conditional analysis $\mathcal{A}^{(\top,\{(l,l')\})}$ analyzes all possible input values for the set of paths containing the true branch of l while $\mathcal{A}^{(\top,\{\neg(l,l')\})}$ does it for the set of paths containing the false branch of l. Obviously, the cumulative result of $\mathcal{A}^{(\top,\{(l,l')\})}$ and $\mathcal{A}^{(\top,\{\neg(l,l')\})}$ is unconditional. To automate the process of combining the set of paths we convert π into a boolean function g_π as follows. Each true edge in CFG_P is mapped to a boolean variable x_i and each false edge is mapped to $\neg x_i$. Then edges in π are mapped to a set of literals, i.e., variables and its negations, and g_π is expressed as the conjunction of those literals. In our example if (l, l') is mapped to x_5 then $g_{\{(ll')\}} := x_5$ and $g_{\{\neg(ll')\}} := \neg x_5$. The union of these two sets of paths is equivalent of the disjunctions of $g_{(ll')}$ and $g_{\neg(ll')}$. Thus the combination of arbitrary π_1 and π_2 is equivalent to

$$g_{\pi_1} \vee g_{\pi_2} \equiv \pi_1 \cup \pi_2$$

Combing two conditions with different predicate condition and the same paths like (ϕ_1, π) and (ϕ_2, π) is more straightforward. Since for input variables x_i and $d_i \in D_{\mathcal{A}}$

$$\phi_1 = (x_1 = d_1, \ldots, x_n = d_n)$$

and

$$\phi_2 = (x_1 = d_{n+1}, \ldots, x_n = d_{2n})$$

then the combination of ϕ_1 and ϕ_2 is

$$(x_1 = d_1 \sqcup d_{n+1}, \ldots, x_n = d_n \sqcup d_{2n}) \equiv \phi_1 \sqcup \phi_2$$

The criterion for which the set of conditional analyses equivalent to unconditional result is when the combination of their conditions returns the tuple $(\top, true)$. To combine the set of conditions $\{(\phi_i, \pi_j)\}$ where $\phi_i \in \Phi$ and $\pi_j \in \Pi$ we first partition them into subsets with the same π_j

$$\{(\phi_i, \pi_j)\} = \{(\phi_i, \pi_1)\} \cup \cdots \cup \{(\phi_i, \pi_n)\}$$

and combine ϕ as described above

$$\{\sqcup_i\{(\phi_i, \pi_1)\}, \ldots, \sqcup_i\{(\phi_i, \pi_n)\}\} = \{(\phi_{i_1}, \pi_1), \ldots, (\phi_{i_n}, \pi_n)\}$$

where $\phi_{i_j} \in \Phi$ and not necessary unique. Next we separate the result above into subsets with the same ϕ_{i_j}

$$\cup_j\{(\phi_1, \pi_j)\} \cup \cdots \cup \cup_j\{(\phi_m, \pi_j)\}$$

and union the paths in each partition by converting them to boolean functions g_π and mapping them back to the set representation

$$\{\vee_j\{(\phi_1, g_{\pi_j})\}, \ldots, \vee_j\{(\phi_m, g_{\pi_j})\}\} = \{(\phi_1, \pi_{j_1}), \ldots, (\phi_m, \pi_{j_m})\}$$

In the case when

$$\{(\phi_1, \pi_{j_1}), \ldots, (\phi_m, \pi_{j_m})\} = \{(\top, \emptyset)\}$$

then the result is unconditional.

IV. Applications of Conditional Analysis

What Elena described about two programs and two analyses

If possible we should introduce the key research questions early in the paper. As I said earlier, if we can provide partial answers to the questions via analytical argument that is best.

V. Related Work

In the introduction we have highlighted the key concepts presented in the previous publications on conditional analyses. In this section we describe most relevant to our work ideas from those papers. The earlier research has introduced two "conditional" notions – *conditional soundness* and *conditional analysis*. In their essence both concepts are the same – the result of an analysis holds only under the established conditions. They diverge in their approaches in obtaining unconditional results. Conditional soundness relies on another analysis to prove that the program states that satisfy the negated condition are not reachable, i.e., showing the infeasibility of the program execution containing those states. While conditional analysis may use the same or a different analysis to show that those states do not violate properties either by appearing in behaviors that conform to the properties or by belonging to infeasible program executions.

The work on conditional soundness [6] explores the dependency of the soundness of one analysis on the result of another analysis. Many analyses assume that a program is "well-behaved", for example an analysis with the abstract domain $\{-, 0, +\}$ assumes the absence of overflow conditions when the rule $\{+\} + 1$ produces not expected $\{+\}$ but $\{-\}$. The authors demonstrate this concept on a pointer analysis for C program. A points-to analysis can produce sound results if the program is memory safe, i.e., there is no out of bound accesses. The authors argue that it is possible to combine points-to and memory safe analyses into a single analysis but such approach is not scalable. Thus they propose to first perform points-to analysis assuming that the program is memory safe and after that run the memory safety analysis with the points-to analysis result to prove that the assumption holds.

Besides explaining the concept of conditional soundness the authors also formalize it through a set of definitions. They specify of a conditional soundness with respect to a predicate θ on a concrete state. Then an analysis is defined to be θ sound when it over-approximates program behavior reachable through concrete states satisfying θ. Thus an analysis applies its transfer function only to the states satisfying θ. However, the transfer function can produce non-θ states but those states will not be explored on the next application of the transfer function. In order to prove an unconditional soundness of the first analysis the second analysis explores the states produced by the first analysis and proves that resulted non-θ states are unreachable in the program.

The work [5] on static race detection for multi-threaded programs is conceptually similar to the previously described paper on conditional soundness. A static race detection algorithm analyzes memory accesses m_1 and m_2 guarded by locks l_1 and l_2 respectively. It assumes that m_1 and m_2 may alias and thus the algorithm aims to prove that l_1 and l_2 must alias in order for a program to be race free. In the alternative approach proposed in this paper it is assumed that l_1 and l_2 must not alias and then it is shown that m_1 and m_2 cannot refer to the same memory location. The absence of aliasing between m_1 and m_2 is easy to show when m_i is the field of the object referred by l_i and that the field m_i is only reachable through that object. However, in order to make the result of the analysis sound unconditionally another analysis must be prove that l_1 and l_2 in fact must not alias.

The authors of the paper on conditional model checking [7] emphasize the fact that when a model checker fails either to verify a program or to produce a counterexample then the effort put into the exploration of the program's state space is wasted. These situations arise when either the model checker runs out of computational resources or its unable to decide on the predicate of a branch condition. The latter takes place when the model checker is not designed to handle the theory used in the predicate, e.g., the predicate analysis cannot handle non-linear arithmetic.

In the case of resources exhaustion, i.e., either running out of memory or timing out, instead of conventional termination the authors propose an informative termination where upon shutting down the model checker produces data that describes the parts of the state space it was able to verify. To characterize the parts of the analyzed state space the model checker enhances each abstract state with a predicate that uniquely describes that state. Hence, when the model checker depletes its computational resources, instead of terminating with a run-time exception, it returns a condition describing the set of states it has analyzed. The condition is expressed as a conjunction of the analyzed states predicates. The next model checker can use the negated condition to guide itself to the part of the state space that was not analyzed by the previous model checker. This characteristic of conditional model checker corresponds to the notion of conditional analysis.

In the case of undecidable predicates the authors propose to proceed with such predicate interpretation that permits the model checker to proceed with the verification, e.g., assuming that an assertion it cannot reason about does hold. The encountered assumptions for taken branches are encoded as an automaton that is used to guide the further analysis to the assumed branches. These assumptions must be analyzed further by another model checker that is capable to reason about them. If the subsequent model checker determines that these assumptions hold then the verification is complete. This side of conditional model checker relates to the idea of conditional soundness.

A conditional model checker integrates assumptions, i.e., conditional soundness for certain states, and predicate , i.e., conditional analysis, information into the characteristic of abstract states. At the end of the execution the conditional model checker traverses the reached abstract states and produces the condition and the branch outcome assumption information. The experiments presented in the paper have shown that applying conditional model checkers with feedback capabilities

increased the number of verified programs.

Our work differs from the previous research in the approach of determining the condition for which an analysis guarantees its most accurate result. In the work on conditional soundness these conditions are defined a priori and applied for all states, e.g., no integer overflows. These conditions are derived straight from the design of the analysis. Without those conditions in place the analysis is unable to produce a correct answer. A conditional model checker discovers the conditions, i.e., the assumptions, for its optimal result while analyzing a program. Although its conditions only applicable to some set of states, e.g., the false branch of the conditional statement on line 10 is infeasible.

However, conditional data-flow analysis cannot use neither of these approaches. Clearly, finding the conditions solely from the design of data-flow analysis is not possible because these conditions are program dependent. Therefore no a priori conditions can be determined. Determining them during analysis run as done in the conditional model checking is also problematic. The reason is that model checking and data-flow analysis have different underlying analysis frameworks. Model checking is based on meet over path (MOP) analysis framework where an analysis calculates facts for each program path separately. While data flow analysis is based on maximal fixed point (MFP) analysis framework where only a single fact is calculated for a set of paths and that fact over-approximates the facts of the evaluated paths.

Thus, when an MOP analysis cannot decide on a branch outcome it is solely due to the analysis theory being inadequate. However, for an MFP analysis an imprecision may also be the result of the over-approximating several paths with diverse facts. But the MFP analysis might produce a better result if the set of paths it over-approximates is smaller with more uniform facts. The current work looks into exploring alternative methods in partitioning program paths to increase the precision of data flow analysis.

VI. Conclusion and Future Work

Acknowledgment

The authors would like to thank...

References

[1] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, ser. POPL '77. New York, NY, USA: ACM, 1977, pp. 238–252. [Online]. Available: http://doi.acm.org/10.1145/512950.512973

[2] ——, "Systematic design of program analysis frameworks," in *Proceedings of the 6th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, ser. POPL '79. New York, NY, USA: ACM, 1979, pp. 269–282. [Online]. Available: http://doi.acm.org/10.1145/567752.567778

[3] ——, "Modular static program analysis," in *Proceedings of the 11th International Conference on Compiler Construction*, ser. CC '02. London, UK, UK: Springer-Verlag, 2002, pp. 159–178. [Online]. Available: http://dl.acm.org/citation.cfm?id=647478.727794

[4] C. Ballabriga, H. Cass, and P. Sainrat, "Wcet computation on software components by partial static analysis," in *In Junior Researcher Workshop on Real-Time Computing*, 2007, pp. 15–18.

[5] M. Naik and A. Aiken, "Conditional must not aliasing for static race detection," in *Proceedings of the 34th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, ser. POPL '07. New York, NY, USA: ACM, 2007, pp. 327–338. [Online]. Available: http://doi.acm.org/10.1145/1190216.1190265

[6] C. L. Conway, D. Dams, K. S. Namjoshi, and C. Barrett, "Pointer analysis, conditional soundness, and proving the absence of errors," in *Proceedings of the 15th international symposium on Static Analysis*, ser. SAS '08. Berlin, Heidelberg: Springer-Verlag, 2008, pp. 62–77. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-69166-2_5

[7] D. Beyer, T. A. Henzinger, M. E. Keremoglu, and P. Wendler, "Conditional model checking," *CoRR*, vol. abs/1109.6926, 2011.

[8] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Secaucus, NJ, USA: Springer-Verlag New York, Inc., 1999.